

Chap01-Introduction

1.1 Stable Matching

稳定匹配对

Def.(1-2). A **perfect matching** S' is a matching with the property that each member of M and each member of W appears in *exactly* one pair of S' .

不稳定匹配对

Def.(1-3). Given a perfect matching S , man m and woman w' form an **unstable pair w.r.t. S** if both:

- m prefers w' to matched woman w in S .
- w prefers m' to matched man m in S .

Gals-Shapley Algorithms

```
Initialize each person to be free.
while (some man is free and hasn't proposed to every woman) {
    Choose such a man m
    w = 1st woman on m's list to whom m has not yet proposed
    if (w is free)
        assign m and w to be engaged
    else if (w prefers m to her fiancé m')
        assign m and w to be engaged, and m' to be free
    else
        w rejects m
}
```

时间复杂度 $O(n^2)$

有效匹配

Def.(1-5) A women w is a **valid partner** of a man m if there exists a stable matching that contains the pair (m, w) .

E.g. Both X and Y are valid partners for A.

	1 st	2 nd	3 rd		1 st	2 nd	3 rd
A	X	Y	Z	X	B	A	C
B	Y	X	Z	Y	A	B	C
C	X	Y	Z	Z	A	B	C

an instance with two stable matchings: $S = \{ A-X, B-Y, C-Z \}$ and $S' = \{ A-Y, B-X, C-Z \}$

最优有效匹配

Def.(1-6) A woman w is a **best valid partner** of a man m if w is a valid partner of m , and no woman whom m ranks higher than w is a valid partner of his.

1.2 Five Representative Problems

区间规划

时间复杂度 $O(n \log n)$

带权区间规划

时间复杂度 $O(n \log n)$

二分图匹配

时间复杂度 $O(n^k)$

独立集

NP完全问题

工厂选址竞争

PSPACE完全性问题

Chap02-Basics of Algorithm Analysis

2.1 Computational Tractability

若一个算法有多项式级别的运行时间，则称其为有效算法。

2.2 Asymptotic Order of Growth

渐进上界（可有多多个）

- $f(n)$ is $O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

- $$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

渐进下界（可有多多个）

- $f(n)$ is $\Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $f(n) \geq c \cdot g(n) \geq 0$ for all $n \geq n_0$

- $$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

渐进紧界（只有一个）

- $f(n)$ is $\Theta(g(n))$ if there exist constants $c_1 > 0$, $c_2 > 0$ and $n_0 \geq 0$ such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$

- $$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$$

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

2.3 Survy of Common Running Times

二分查找

$lo \leftarrow 1; hi \leftarrow n.$



WHILE ($lo \leq hi$)

$mid \leftarrow \lfloor (lo + hi) / 2 \rfloor.$

IF ($x < A[mid]$) $hi \leftarrow mid - 1.$

ELSE IF ($x > A[mid]$) $lo \leftarrow mid + 1.$

ELSE RETURN $mid.$

RETURN $-1.$

时间复杂度 $O(\log n)$

归并排序

$i \leftarrow 1; j \leftarrow 1.$

WHILE (both lists are nonempty)

IF ($a_i \leq b_j$) append a_i to output list and increment $i.$

ELSE append b_j to output list and increment $j.$

Append remaining elements from nonempty list to output list.

时间复杂度 $O(n)$

Chap03-Graphs

3.1 Basic Definitions and Applications

3.2 Graph Traversal and Graph Connectivity

Breadth-First Search

```
1 Initialize visited and visited[s] = true
2 for v in V-s:
3     visited[v] = false
4 End for
5 Initialize L and add s into L[0]
6 i=0
7 Initialize T
8 while L[i] is not empty:
9     Initialize L[i]
10    for u in L[i]
11        select (u,v) in E
12        if visited(v) == false:
13            visited(v) = true
14            add (u,v) into T
15            add v into L[i+1]
16        end if
17    end for
18    i=i+1
19 end while
```

时间复杂度 $O(m + n)$

Depth-First Search

```
1 bool visited[MaxVertexNum] //存储访问状态的数组
2 void DFSTraverse(Graph G){
3     int v;
4     for(v=0;v<G.vexnum;++v) //遍历所有顶点
5         visited[v]=FALSE; //将所有顶点初始化为未访问状态
6 }
```

```

7     for(v=0;v<G.vexnum;++v) //遍历所有顶点
8         if(!visited[v]) //找到第一个FALSE顶点
9             DFS(G,v); //对其进行DFS遍历
10    }
11    void DFS(Graph G,int v) //递归遍历，顶点v、图G
12    {
13        visit(v); //访问顶点v
14        visited[v]=true; //将判断数组置为已访问
15
16        for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))
17            if(!visited[w]){
18                DFS(G,w);
19            } //if
20    }

```

时间复杂度 $O(m+n)$

连通测试

R will consist of nodes to which s has a path
Initially $R = \{s\}$
While there is an edge (u, v) where $u \in R$ and $v \notin R$
Add v to R
Endwhile



3.3 Testing Bipartiteness

图是二分的，当且仅当图里没有奇数长度的环

```

1  void hungary() //匈牙利算法
2  {
3      for i->1 to nx //对于集合X中的每一个节点
4          if (从未匹配点i出发有增广路)
5              匹配数++;
6      输出 匹配数;
7  }
8  bool findpath(x) //寻找从x出发的对应项出的可增广路
9  {
10     //crossPath[x] = true;这条语句可能会有很多人加上，但是实际上crossPath总是记录集合Y中的节点是否在交错路上
11     for each edge(x,y) in G.E
12     {
13         if(y is not in crossPath)
14         {
15             add y into crossPath
16             lastx = match[y]; //lastx是X集合的上一个与y匹配的节点
17             if(y is not matched or findpath(lastx)) //如果y已经被了，那么试试从lastx能不能另外找到一条增广路，把当前增广路让给现
18                 在的x
19                 {
20                     match[y] = x;
21                     //match[x] = y, wrong !
22                     return true; //从x出发有增广路
23                 }
24         }
25     }
26     return false; //从x出发没有增广路
27 }

```

3.4 Connectivity in Directed Graphs

- Pick any node s .
- Run BFS from s in G .
- Run BFS from s in G^{rev} .
- Return true iff all nodes reached in both BFS executions.
- Correctness follows immediately from previous lemma.

时间复杂度 $O(m + n)$

3.5 DAGs and Topological Ordering

如果一个图有拓扑排序，当且仅当图是有向无环图

```

To compute a topological ordering of  $G$ :
Find a node  $v$  with no incoming edges and order it first
Delete  $v$  from  $G$ 
Recursively compute a topological ordering of  $G - \{v\}$ 
and append this order after  $v$ 

```

时间复杂度 $O(m + n)$

Chap04-Greedy Algorithms

- 贪心选择性
- 最优子结构

4.1 Interval Scheduling and Interval Partitioning

Interval Scheduling

Earliest-Finish-Time-First(EFTF) algorithm

EARLIEST-FINISH-TIME-FIRST ($n, s_1, s_2, \dots, s_n, f_1, f_2, \dots, f_n$)

SORT jobs by finish times and renumber so that $f_1 \leq f_2 \leq \dots \leq f_n$.

$S \leftarrow \emptyset$. ← set of jobs selected

FOR $j = 1$ **TO** n

IF (job j is compatible with S)

$S \leftarrow S \cup \{j\}$.

RETURN S .

时间复杂度 $O(n \log n)$

Dijkstra's Algorithm

DIJKSTRA (V, E, ℓ, s)

FOREACH $v \neq s$: $\pi[v] \leftarrow \infty, \text{pred}[v] \leftarrow \text{null}; \pi[s] \leftarrow 0$.

Create an empty priority queue pq .

FOREACH $v \in V$: **INSERT**($pq, v, \pi[v]$).

WHILE (**IS-NOT-EMPTY**(pq))

$u \leftarrow \text{DEL-MIN}(pq)$.

FOREACH edge $e = (u, v) \in E$ leaving u :

IF ($\pi[v] > \pi[u] + \ell_e$)

DECREASE-KEY($pq, v, \pi[u] + \ell_e$).

$\pi[v] \leftarrow \pi[u] + \ell_e; \text{pred}[v] \leftarrow e$.

4.5 Minimum Spanning Tree

记C是一个环路，D是一个割集，则C跟D共有偶数条公共边

Prim's Algorithm (已有的边扩展)

PRIM (V, E, c)

$S \leftarrow \emptyset, T \leftarrow \emptyset$.

$s \leftarrow$ any node in V .

FOREACH $v \neq s$: $\pi[v] \leftarrow \infty, \text{pred}[v] \leftarrow \text{null}; \pi[s] \leftarrow 0$.

Create an empty priority queue pq .

FOREACH $v \in V$: **INSERT**($pq, v, \pi[v]$).

WHILE (**IS-NOT-EMPTY**(pq))

$u \leftarrow \text{DEL-MIN}(pq)$.

$\pi[v]$ = cost of cheapest
known edge between v and S

$S \leftarrow S \cup \{u\}, T \leftarrow T \cup \{\text{pred}[u]\}$.

FOREACH edge $e = (u, v) \in E$ with $v \notin S$:

IF ($c_e < \pi[v]$)

DECREASE-KEY(pq, v, c_e).

$\pi[v] \leftarrow c_e; \text{pred}[v] \leftarrow e$.

时间复杂度 $O(m \log n)$

Kruskal's Algorithm (全图找最短消耗)

按照边的消耗排序

```

Kruskal(G, c) {
  Sort edges weights so that  $c_1 \leq c_2 \leq \dots \leq c_m$ .
   $T \leftarrow \phi$ 

  foreach (u  $\in$  V) make a set containing singleton u

  for i = 1 to m    are u and v in different connected components?
    (u,v) =  $e_i$     ↗
    if (u and v are in different sets) {
       $T \leftarrow T \cup \{e_i\}$ 
      merge the sets containing u and v
    }                ↖ merge two components
  return T
}

```

时间复杂度 $O(m \log n)$

4.7 Clustering

4.8 Huffman Codes

```

Huffman(S) {
  if |S|=2 {
    return tree with root and 2 leaves
  } else {
    let y and z be lowest-frequency letters in S
     $S' = S$ 
    remove y and z from  $S'$ 
    insert new letter  $\omega$  in  $S'$  with  $f_\omega = f_y + f_z$ 
     $T' = \text{Huffman}(S')$ 
    T = add two children y and z to leaf  $\omega$  from  $T'$ 
    return T
  }
}

```

Chap05-Divide and Conquer

5.1 Mergesort Algorithm

MERGE-SORT(L)

IF (list L has one element)

RETURN L.

Divide the list into two halves A and B.

$A \leftarrow \text{MERGE-SORT}(A)$. $\leftarrow T(n/2)$

$B \leftarrow \text{MERGE-SORT}(B)$. $\leftarrow T(n/2)$

$L \leftarrow \text{MERGE}(A, B)$. $\leftarrow \Theta(n)$

RETURN L.

时间复杂度 $O(n \log_2 n)$

5.2 Recurrence Relations

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^c)$$

$$T(n) = \begin{cases} \Theta(n^c) & \text{if } c > \log_b a \\ \Theta(n^c \log n) & \text{if } c = \log_b a \\ \Theta(n^{\log_b a}) & \text{if } c < \log_b a \end{cases}$$

5.3 Counting Inversions

SORT-AND-COUNT(L)

IF (list L has one element)

RETURN $(0, L)$.

Divide the list into two halves A and B .

$(r_A, A) \leftarrow \text{SORT-AND-COUNT}(A)$. $\leftarrow T(n/2)$

$(r_B, B) \leftarrow \text{SORT-AND-COUNT}(B)$. $\leftarrow T(n/2)$

$(r_{AB}, L) \leftarrow \text{MERGE-AND-COUNT}(A, B)$. $\leftarrow \Theta(n)$

RETURN $(r_A + r_B + r_{AB}, L)$.

时间复杂度 $O(n \log n)$

5.4 Finding the Closest Pairs of Points

CLOSEST-PAIR($p_1, p_2, \dots, p_n$)

Compute vertical line L such that half the points are on each side of the line.

$\delta_1 \leftarrow \text{CLOSEST-PAIR}(\text{points in left half})$.

$\delta_2 \leftarrow \text{CLOSEST-PAIR}(\text{points in right half})$.

$\delta \leftarrow \min \{ \delta_1, \delta_2 \}$.

Delete all points further than δ from line L .

Sort remaining points by y-coordinate.

Scan points in y-order and compare distance between each point and next 7 neighbors. If any of these distances is less than δ , update δ .

RETURN δ .

时间复杂度 $O(n \log^2 n)$

$\leftarrow O(n)$
 $\leftarrow T(n/2)$
 $\leftarrow T(n/2)$
 $\leftarrow O(n)$
 $\leftarrow O(n \log n)$
 $\leftarrow O(n)$

Chap06-Dynamic Programming

6.1 矩阵链相乘

使用动态规划法:

$$M_1 \cdot M_2 \cdot M_3 \cdots M_n$$

$$r_1, r_2, r_3, \dots, r_n, r_{n+1}$$

$$(M_i)_{r_i \times r_{i+1}} \quad 1 \leq i \leq n$$

$$M_{i,j} = M_i \cdot M_{i+1} \cdots M_{j-1} \cdot M_j$$

$C[i, j]$: 计算 $M_{i,j}$ 所需的最小乘法次数。

$i = 1, j = n$ 时, 原问题得解。

$$M_{i,j} = (M_i \cdot M_{i+1} \cdots M_{k-1}) \cdot (M_k \cdot M_{k+1} \cdots M_{j-1} \cdot M_j)$$

$C[i, j]$ $C[i, k-1]$ $C[k, j]$

$$C[i, j] = C[i, k-1] + C[k, j] + r_i \cdot r_k \cdot r_{j+1}$$
$$C[i, j] = \min_{i < k \leq j} \{ C[i, k-1] + C[k, j] + r_i \cdot r_k \cdot r_{j+1} \}$$

$k = i+1$ $k = j$

输入: $r[1..n+1]$, 表示 n 个矩阵规模的 $n+1$ 个整数.

输出: n 个矩阵连乘的最小乘法次数.

```

1. for  $i \leftarrow 1$  to  $n$  {填充对角线  $d_0$ }
2.    $C[i,i] \leftarrow 0$ 
3. end for
4. for  $d \leftarrow 1$  to  $n-1$  {填充对角线  $d_1$  到  $d_{n-1}$ }
5.   for  $i \leftarrow 1$  to  $n-d$  {填充对角线  $d_i$  的每个项目}
6.      $j \leftarrow i+d$  {该对角线上  $j,i$  满足的关系}
7.      $C[i,j] \leftarrow \infty$ 
8.     for  $k \leftarrow i+1$  to  $j$ 
9.        $C[i,j] \leftarrow \min\{C[i,j], C[i,k-1] + C[k,j] + r_i \times r_k \times r_{j+1}\}$ 
10.    end for
11.  end for
12. end for
13. return  $C[1,n]$ 

```

$$T(n) = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{k=i+1}^j c = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{k=i+1}^{i+d} c = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{k=1}^d c = \Theta(n^3)$$

一个实例

$(M_1)_{5 \times 10} \cdot (M_2)_{10 \times 4} \cdot (M_3)_{4 \times 6} \cdot (M_4)_{6 \times 10} \cdot (M_5)_{10 \times 2}$
 $r_1 = 5, r_2 = 10, r_3 = 4, r_4 = 6, r_5 = 10, r_6 = 2$

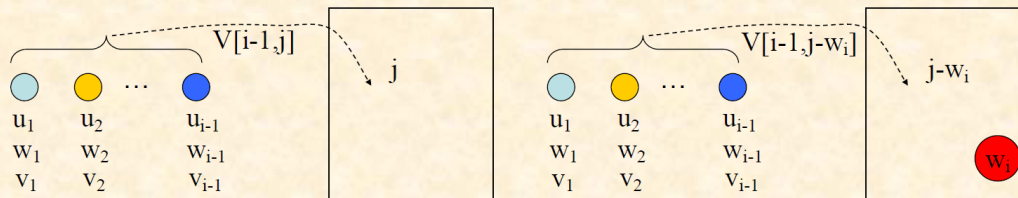
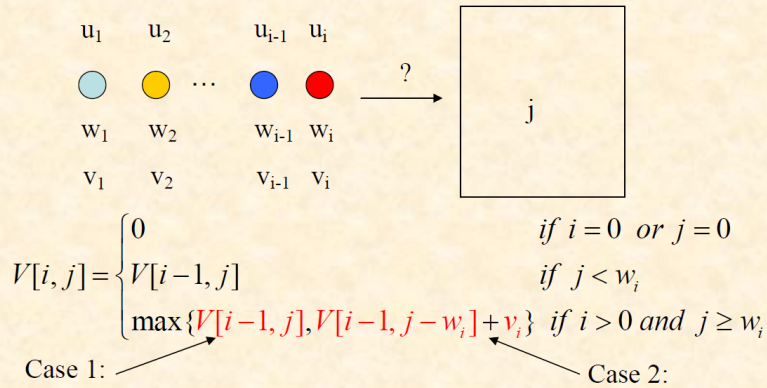
$d=0$	$d=1$	$d=2$	$d=3$	$d=4$
$C[1,1]=0$ (M_1)	$C[1,2]=200$ ($M_1 M_2$)	$C[1,3]=320$	$C[1,4]=620$	$C[1,5]=348$
	$C[2,2]=0$ (M_2)	$C[2,3]=240$ ($M_2 M_3$)	$C[2,4]=640$ (M_2) ($M_3 M_4$)	$C[2,5]=248$
		$C[3,3]=0$ (M_3)	$C[3,4]=240$ ($M_3 M_4$)	$C[3,5]=168$
			$C[4,4]=0$ (M_4)	$C[4,5]=120$ ($M_4 M_5$)
				$C[5,5]=0$ (M_5)

$$C[2,4] = \min_{2 \leq k \leq 4} \{C[2, k-1] + C[k, 4] + r_2 \cdot r_k \cdot r_{4+1}\}$$

$$\begin{aligned}
k=3 &\rightarrow C[2,4] = C[2,2] + C[3,4] + r_2 \cdot r_3 \cdot r_{4+1} = 0 + 240 + 10 \cdot 4 \cdot 10 = 640 \rightarrow (M_2) \cdot (M_3 \cdot M_4) \\
k=4 &\rightarrow C[2,4] = C[2,3] + C[4,4] + r_2 \cdot r_4 \cdot r_{4+1} = 240 + 0 + 10 \cdot 6 \cdot 10 = 840 \rightarrow (M_2 \cdot M_3) \cdot (M_4)
\end{aligned}
\} \min$$

6.2 0-1背包问题

设 $V[i,j]$ 表示从前 i 个物品 $\{u_1, u_2, \dots, u_i\}$ 中取出一部分装入承重为 j 的背包所能取得的最大价值。那么，当 $i=n, j=C$ 时， $V[n,C]$ 就是原问题的解。



输入：物品集合 $\{u_1, u_2, \dots, u_n\}$ ，重量分别为 $w_1, w_2, \dots, w_n$ ，价值分别为 $v_1, v_2, \dots, v_n$ ，承重为 C 的背包

输出：背包所能装物品的最大价值

```

1. for  $i \leftarrow 0$  to  $n$ 
2.    $V[i, 0] \leftarrow 0$ 
3. end for
4. for  $j \leftarrow 0$  to  $C$ 
5.    $V[0, j] \leftarrow 0$ 
6. end for
7. for  $i \leftarrow 1$  to  $n$  //前 $i$ 个物品
8.   for  $j \leftarrow 1$  to  $C$  //承重 $C$ 与物品重量 $w_i$ 均为整数，故 $j$ 为整数
9.      $V[i, j] \leftarrow V[i-1, j]$ 
10.    if  $w_i \leq j$  then  $V[i, j] \leftarrow \max\{V[i, j], V[i-1, j-w_i] + v_i\}$ 
11.    end if
12.  end for
13. end for
14. return  $V[n, C]$ 

```

$$T(n) = \Theta(nC)$$

一个实例

背包的承重量为 $C=9$ ；给定4个物品，重量(w)分别为2, 3, 4, 5；价值(v)依次为3, 4, 5, 7。问：背包中最多能装的物品的总价值是对少？

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7
3	0	0	3	4	5	7	8	9	9	12
4	0	0	3	4	5	7	8	10	11	12

因为 $w_3=4 \leq j=7$;
 所以 $V[3, 7] = \max\{V[3-1, 7], V[3-1, 7-w_3]+v_3\}$
 $= \max\{V[2, 7], V[2, 3]+5\} = \max\{7, 4+5\} = 9$

6.3 最长公共子序列问题

$$A = a_1 a_2 \cdots a_n \quad B = b_1 b_2 \cdots b_m$$

$$\left. \begin{array}{l} a_1 a_2 \cdots a_{i-1} a_i \\ b_1 b_2 \cdots b_{j-1} b_j \end{array} \right\} \xrightarrow{\text{最长公共子序列的长度值}} C[i, j]$$

$$C[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ C[i-1, j-1] + 1 & \text{if } i>0 \& j>0 \& a_i == b_j \\ \max\{C[i, j-1], C[i-1, j]\} & \text{if } i>0 \& j>0 \& a_i \neq b_j \end{cases}$$

$$\begin{array}{l} \underline{a_1 a_2 \cdots a_{i-1} a_i} \\ \underline{b_1 b_2 \cdots b_{j-1} b_j} \end{array} \xrightarrow{\quad} C[i, j-1]$$

$$\begin{array}{l} \underline{a_1 a_2 \cdots a_{i-1} a_i} \\ \underline{b_1 b_2 \cdots b_{j-1} b_j} \end{array} \xrightarrow{\quad} C[i-1, j]$$

max

输入：两个字符串A, B, 长度分别为n, m.

输出：X和Y的最长公共子序列长度.

```
1. for i ← 0 to n
2.   C[i,0] ← 0
3. end for
4. for j ← 0 to m
5.   C[0,j] ← 0
6. end for
7. for i ← 1 to n
8.   for j ← 1 to m
9.     if  $a_i=b_j$  then  $C[i, j] \leftarrow C[i-1, j-1]+1$ 
10.    else  $C[i, j] \leftarrow \max \{C[i, j-1], C[i-1, j]\}$ 
11.    end if
12.   end for
13. end for
14. return C[n,m]
```

$$T(n) = \Theta(nm)$$

一个实例：

$$C[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ C[i-1, j-1]+1 & \text{if } i>0 \& j>0 \& a_i = b_j \\ \max \{C[i, j-1], C[i-1, j]\} & \text{if } i>0 \& j>0 \& a_i \neq b_j \end{cases}$$

$A = xyxxz \quad B = zxzyyz$

使用一个 $(n+1) \times (m+1)$ 的表格来进行计算。

逐行填满表格，问题得解。

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
2	0	0	1	1	2	2	2
3	0	0	1	1	2	2	2
4	0	0	1	1	2	2	2
5	0	1	1	2	2	2	3

因为 $a_3 \neq b_4$, 所以 $C[3, 4] = \max \{C[3, 3], C[2, 4]\} = \max \{1, 2\} = 2$

因为 $a_5 = b_6$, 所以 $C[5, 6] = C[4, 5] + 1 = 2 + 1 = 3$

6.4 整数划分问题

- 在正整数K的所有划分中，我们将最大加数(k_1) 不大于(小于或等于) m的划分数记做 $p(K, m)$
- 那么当 $m=K$ 时，就是问题的解。
- 例如 $p(6, 1) = 1$; $p(6, 2) = 4$; $p(6, 3) = 7$; $p(6, 4) = 9$; $p(6, 5) = 10$; $p(6, 6) = 11$

$$p(K, m) = \begin{cases} 1, & \text{if } (m = 1) \\ p(K, K), & \text{if } (m > K) \\ 1 + p(K, K - 1), & \text{if } (m = K) \\ p(K - m, m) + p(K, m - 1), & \text{if } (1 < m < K) \end{cases}$$

Algorithm	partitionNum(K,m)
1	For i = 1 to K
2	p[i,1] = 1; p[1,i] = 1;
	End for
3	For j = 2 to m
4	For i = 2 to K
	If i < j
	p[i,j] = p[i,i];
5	If i == j
6	p[i,j] = 1 + p[i,i-1];
7	If i > j
	p[i,j] = p[i, j-1] + p[i-j, j];
	End for
	End for
8	Return p[K,m]

Chap07-Backtracking

递归回溯

输入：集合 $X_1, X_2, \dots, X_n$ 清楚或是隐含的描述

输出：解向量 $v = (x_1, x_2, \dots, x_n)$, $0 \leq i \leq n$

1. $v \leftarrow ()$
2. $\text{flag} \leftarrow \text{false}$
3. $\text{advance}(1)$
4. if flag then output v
5. else output “no solution”

过程 $\text{advance}(k)$

1. for 每个 $x \in X_k$
2. $x_k \leftarrow x$; 将 x_k 加入v
3. if c 为最终解 then set $\text{flag} \leftarrow \text{true}$ and exit
4. else if v是部分解 then $\text{advance}(k+1)$
5. end for

迭代回溯

输入：集合 $X_1, X_2, \dots, X_n$ 清楚或是隐含的描述

输出：解向量 $v = (x_1, x_2, \dots, x_n)$, $0 \leq i \leq n$

```
1.  $v \leftarrow ()$ 
2.  $\text{flag} \leftarrow \text{false}$ 
3.  $k \leftarrow 1$ 
4. while  $k \geq 1$ 
5.   while  $X_k$  没有被穷举
6.      $x_k \leftarrow X_k$  里的下一个元素；将  $x_k$  加入  $v$ 
7.     if  $v$  为最终解 then set  $\text{flag} \leftarrow \text{true}$  and exit
8.     else if  $v$  是部分解 then  $k \leftarrow k+1$  //前进
9.   end while
10. 重置  $X_k$  //一般将  $x_k \leftarrow 0$ ，表示将其重置
11.  $k \leftarrow k-1$  //回溯
12. end while
13. if  $\text{flag}$  then output  $v$ 
14. else output "no solution"
```

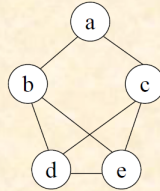
7.1 图的三着色问题

递归回溯

输入：无向图 $G=(V,E)$

输出：G的顶点的3着色 $c[1 \dots n]$ ，其中每个 $c[j]$ 为1,2或3.

```
1. for  $k \leftarrow 1$  to  $n$ 
2.    $c[k] \leftarrow 0$  //no color
3. end for
4.  $\text{flag} \leftarrow \text{false}$ 
5.  $\text{graphcolor}(1)$ 
6. if  $\text{flag}$  then output  $c$ 
7. else output "no solution"
```



$\text{graphcolor}(k)$

```
1. for color=1 to 3
2.    $c[k] \leftarrow \text{color}$ 
3.   if  $c$  为合法着色 then  $\text{flag} \leftarrow \text{true}$  and exit
4.   else if  $c$  是部分着色 then  $\text{graphcolor}(k+1)$ 
5. end for
```

迭代回溯

输入：无向图 $G=(V,E)$

输出：G的顶点的3着色 $c[1 \dots n]$ ，其中每个 $c[j]$ 为1,2或3.

```
1. for  $k \leftarrow 1$  to  $n$ 
2.    $c[k] \leftarrow 0$ 
3. end for
4.  $\text{flag} \leftarrow \text{false}$ 
5.  $k \leftarrow 1$  //start from  $v_1$ 
6. while  $k \geq 1$ 
7.   while  $c[k] \leq 2$  //为第k个顶点着色
8.      $c[k] \leftarrow c[k]+1$ 
9.     if  $c$  为合法着色 then  $\text{flag} \leftarrow \text{true}$  and exit
10.    else if  $c$  为部分着色 then  $k \leftarrow k+1$  //准备为下一顶点着色
11.  end while //假设着色既 not 合法也非部分的，即死节点，试其他颜色
12.  $c[k] \leftarrow 0$  //  $v_k$  试验了所有颜色均失败，当前顶点颜色只好归0，回溯
13.  $k \leftarrow k-1$  //回溯到上一个顶点 (配合第6句的  $k \geq 1$  来理解)
14. end while //注意：回溯到上一个顶点后， $c[k]+1$ ，即尝试下一种颜色
```

7.2 皇后问题

7.3 0-1背包问题

7.4 分支限界法

常见的两种分支限界法

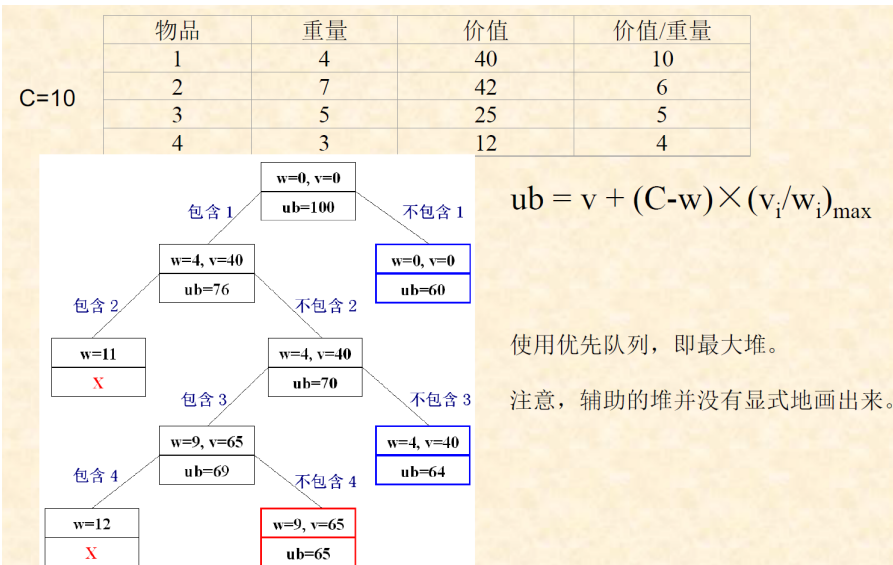
- 从活节点表中选择下一扩展节点的不同方式导致不同的分支限界法：
 - 队列式(FIFO)分支限界法(**宽度优先**)：按照队列先进先出(FIFO)原则选取下一个节点为扩展节点。
 - 优先队列式分支限界法(**最小耗费-[最小堆]或是最大收益优先-[最大堆]**)：按照优先队列中规定的优先级选取优先级最高的节点成为当前扩展节点。
 - 最大优先队列：使用最大堆，体现最大收益优先
 - 最小优先队列：使用最小堆，体现最小耗费优先

0-1背包问题

- 为每个搜索节点设置一个上界ub (upper bound)，一个简单方法是：把已经选择的物品总价值v，加上背包剩余承重量C-w与剩下可选择的物品的最高“价值-重量比”的乘积，即 $ub = v + (C-w) \times (v_i/w_i)_{\max}$
- 比如，若已装物品1，则价值的上界为 $40 + (10-4) \times 6 = 76$

物品	重量	价值	价值/重量
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

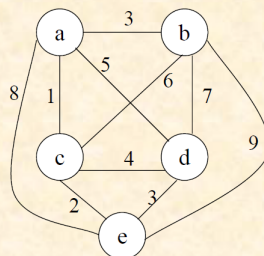
考虑n=4的背包问题（背包承重量为C=10）：



旅行商问题

一个实例

- 考虑如右图所示的TSP问题 (假设每条边上的权值为非负整数)。



- 作为演示，附加一个**特殊的约束条件**：b 在c之前被访问。

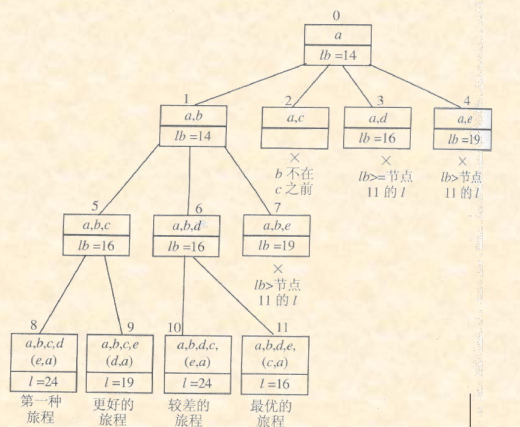
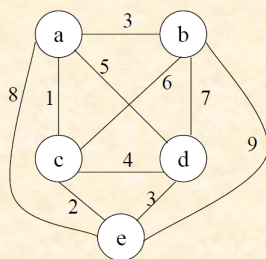
从该节点继续搜索，所获得的收益不会超过ub

0-1背包问题 → 最大收益 → 为搜索节点设置一个上界ub (upper bound) → ub大的节点优先搜索

TSP问题 → 最小耗费 → 为搜索节点设置一个下界lb (lower bound) → lb小的节点优先搜索

从该节点继续搜索，所需要的耗费不会低于lb

搜索树



Chap08-Random Algorithm

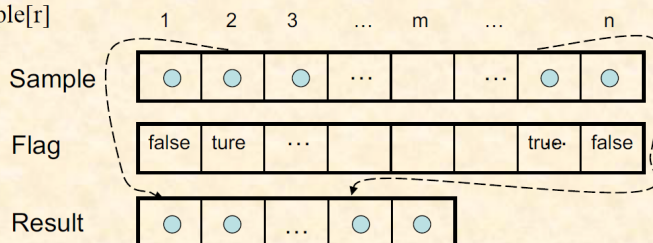
8.1 随机非重复采样问题

输入：样本数组Sample[1...n]，选择样本的个数m，且 $m \leq n/2$

输出：选中的样本Result[1...m]

```

1. for i ← 1 to n
2.   Flag[i] ← false //boolean数组，表示对应的样本是否已被选中
3. end for
4. k ← 0
5. while k < m
6.   r ← random(1,n)
7.   if not Flag[r] then
8.     k ← k+1
9.     Result[k] ← Sample[r]
10.    Flag[r] ← true
11.  end if
12. end while
13. return Result[1...m]
```



时间复杂度 $O(n)$

8.2 快速排序

Algorithm: SPLIT(A[low,...high])

输入：数组A[low,...high]

输出：用A[low]作基准元素划分后的数组A
及基准元素新的位置w

1. $x \leftarrow A[\text{low}]$
2. while (low < high)
3. while (low < high && A[high] > x) --high;
4. $A[\text{low}] \leftarrow A[\text{high}]$
5. while (low < high && A[low] ≤ x) ++low;
6. $A[\text{high}] \leftarrow A[\text{low}]$
7. end while
8. $A[\text{low}] \leftarrow x$
9. $w \leftarrow \text{low}$
10. return A and w //新数组A与x的新位置w

Algorithm: RandomizedQUICKSORT(A[low...high])

输入：n个元素的数组A[low...high]

输出：按非降序排列的数组A[low...high]

1. if low < high then
2. Generate a random integer $r \in [\text{low}, \text{high}]$
3. Exchange A[low] with A[r]
4. $w \leftarrow \text{SPLIT}(A[\text{low} \dots \text{high}])$ {w为基准元素A[low]的新位置}
5. QUICKSORT(A, low, w-1)
6. QUICKSORT(A, w+1, high)
6. end if

时间复杂度 $O(n \log n)$

8.3 测试串的相等性

8.4 模式匹配问题

输入：模式串X，长度为n，模式串Y，长度为m

输出：若Y在X中出现，则返回Y在X中的第1个位置，否则返回-1

1. 从小于M的素数集中随机选择一个素数p
2. $j \leftarrow 1$
3. 计算 $W_p = 2^m \pmod p$, $I_p(Y)$ 和 $I_p(X_1)$ // $O(m) + O(m) + O(m)$
4. while $j \leq n - m + 1$ // $X_j = x_j \dots x_{j+m-1}$, 须有 $j+m-1 \leq n$, 即 $j \leq n-m+1$
5. if $I_p(X_j) = I_p(Y)$ then return j // (可能)找到了匹配子串, $O(\log p)$
6. $I_p(X_{j+1}) \leftarrow (2I_p(X_j) - x_j W_p + x_{j+m}) \pmod p$ // 每步 $O(1)$ 时间, 共 $O(n)$
7. $j \leftarrow j+1$
8. end while
9. return -1 // Y肯定不在X中(若在, 已由第5步返回j值)

时间复杂度分析: $O(m) + O(n) + O(\log p) = O(m+n)$

Chap09-NP Complete Problem

9.1 易解问题与难解问题

- 如果一个问题 Π 存在一个算法，时间复杂性为 $O(n^k)$ ，其中 n 是问题规模， k 是一个非负整数，则称问题 Π 存在**多项式时间**算法。这类算法在**可以接受的时间内**实现问题求解，e.g., 排序、串匹配、矩阵相乘。
- 现实世界里还存在很多问题至今没有找到多项式时间算法，计算时间是指数和超指数函数（如 2^n 和 $n!$ ），随着问题规模的增长而**快速增长**。
- 通常将前者看作是**易解问题**，后者看作**难解问题**。

9.2 P类问题和NP类问题（划分标准是基于对所谓判定问题的求解方式）

确定性算法与P类问题

对判定问题求解，可以采用**确定性算法**

- 定义**9.1(确定性算法)**: 设 A 是求解问题 Π 的一个算法，如果在算法的整个执行过程中，每一步只有一个确定的选择，则称算法 A 是确定性算法。
 - 特点：对同一输入实例，运行算法 A ，所得结果是一样的。
- 定义**9.2(P类问题)**: 如果对于某个判定问题 Π ，存在一个非负整数 k ，对于输入规模为 n 的实例，能够以 $O(n^k)$ 的时间运行一个确定性算法，得到yes或no的答案，则称该判定问题 Π 是一个**P(Polynomial)类问题**。
 - 事实上，所有易解问题都是P类问题。

非确定性算法与NP类问题

- 定义**9.3(非确定性算法)**: 设 A 是求解问题 Π 的一个算法，如果算法 A 以如下**猜测+验证的方式**工作，称算法 A 为非确定性(nondeterminism)算法：
 - **猜测阶段**：对问题的输入实例产生一个任意字符串 y ，在算法的每次运行， y 可能不同，因此猜测是以非确定的形式工作。这个工作一般可以在线性时间内完成。
 - **验证阶段**：在这个阶段，用一个确定性算法验证两件事：首先验证猜测的 y 是否是合适的形式，若不是，则算法停下并回答no；若是合适形式，则继续检查它是否是问题 x 的解，如果确实是 x 的解，则停下并回答yes，否则停下并回答no。要求验证阶段在多项式时间内完成。

- 定义9.4(**NP类问题**): 如果对于判定问题 Π , 存在一个非负整数 k , 对于输入规模为 n 的实例, 能够以 $O(n^k)$ 的时间运行一个非确定性算法, 得到yes/no的答案, 则该判断问题 Π 是一个**NP(nondeterministic polynomial)**类问题。

※注意: **NP**类问题是对于判定问题定义的, 事实上, 可以在多项式时间内应用非确定性算法解决的所有问题都属于**NP**类问题。

P与NP关系

- 1) 若问题 Π 属于**P**类, 则存在一个多项式时间的确定性算法, 对它进行判定或求解; 显然, 也可以构造一个多项式时间的非确定性算法, 来验证解的正确性, 因此, 问题也属**NP**类。因此, 显然有

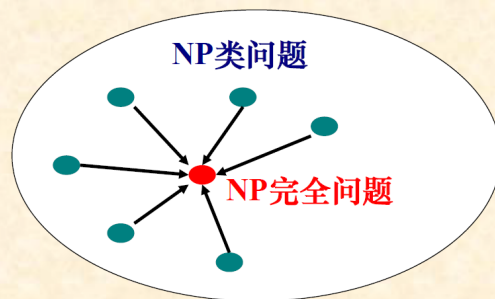
$$P \subseteq NP$$

- 2) 若问题 Π 属于**NP**类, 则存在一个多项式时间的非确定性算法, 来猜测并验证它的解; 但不一定能构造一个多项式时间的确定性算法, 来对它进行求解或判定。

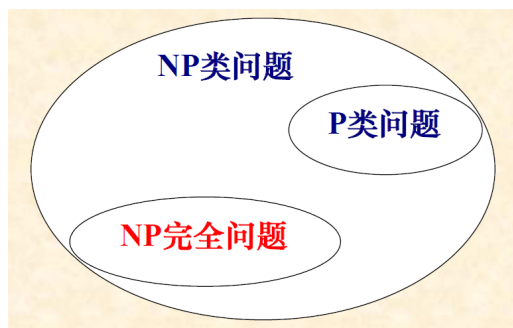
➤ 因此, 人们猜测 **$P \neq NP$** , 但是否成立, 至今未得到证明。

9.3 NP完全问题

- 定义9.7(**NP完全问题**): 令 Π 是一个判定问题, 如果问题 Π 属于**NP**类问题, 并且对**NP**类问题中的每一个问题 Π' , 都有 $\Pi' \leq_{poly} \Pi$, 则称判定问题 Π 是一个**NP完全问题(NP complete problem, NPC)**。



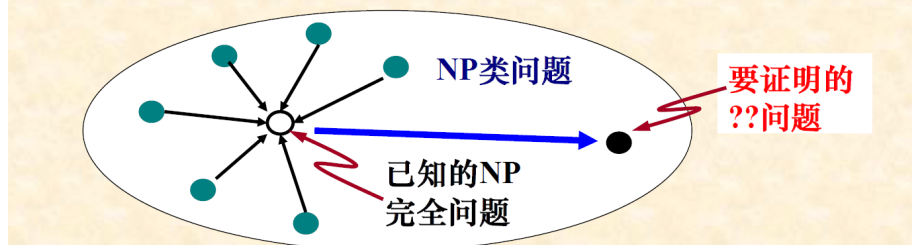
P类问题、NP类问题、NP完全问题的关系



如何证明一个问题是NP完全问题

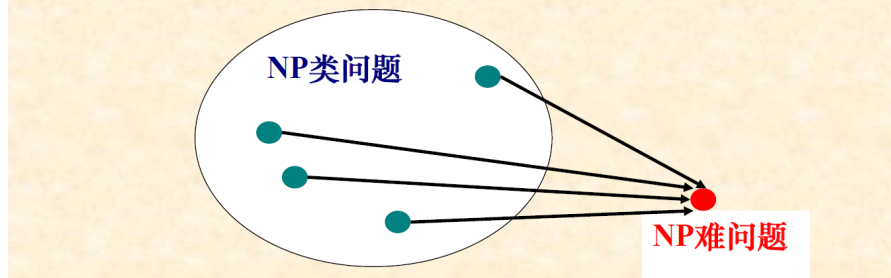
根据NP完全问题的定义（满足的两个性质），显然地，证明需要分两个步骤进行：

- 证明问题 Π 是NP类问题：即可以在多项式时间内以确定性算法验证一个任意生成的串，以确定它是否为问题的一个解。
- 证明NP类问题中的每一个问题都能在多项式时间内变换为问题 Π 。由于多项式问题变换具有传递性，所以，只需证明一个已知的NP完全问题能够在多项式时间内变换为问题 Π 。



9.4 NP难问题

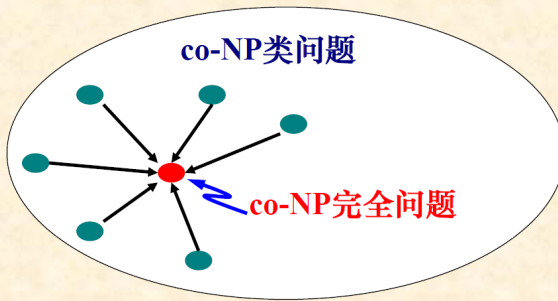
难解问题中还有一类问题，虽然也能证明所有的NP类问题可以在多项式时间内变换到问题 Π ，但并不能证明 Π 也是NP类问题，所以 Π 不是NP完全的。但问题 Π 至少与任意NP类问题有同样的难度，这样的问题称为NP难问题。



9.5 co-NP类问题

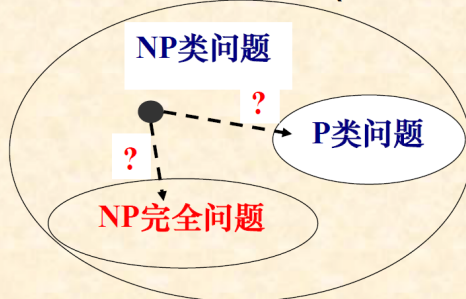
定义9.7(co-NP完全问题): 问题 Π 对于co-NP类是完全的, 若

- Π 在co-NP中;
- 对于co-NP中的每个问题 Π' , 有 $\Pi' \leq_{\text{poly}} \Pi$ 。



9.6 NPI类问题

- NP类问题中还有一些问题, 人们不知道是属于P类还是属于NP完全问题, 还有待于证明其归属。
- 这些问题是NP完全问题的可能性非常小, 也因为不相信他们在P中, 我们人为地增加另一问题类来接纳这类问题, 这个类称为NPI(NP-Intermediate)类。



9.7 P、NP、co-NP、NPI类问题的区别与联系

